文字を数にする

トークン化と語彙 ― ことばを機械に食べさせる下ごしらえ

rkskmt

前回の宿題

- 言語モデルは「次の1文字を当てる」**分類器** ― ここまでは前回の話
- ところで、第1コースで学習に使った材料を思い出すと……
  - → MNIST の画像：28×28 の**数値**（明るさ 0〜1）
  - → 損失も勾配も、ぜんぶ**数値の計算**だった

> **クイズ**
>
> 「あ」は数字ではない。ニューラルネットに「あ」をどうやって入れる？

- 答えはあっけない：**番号を振る**。今日はそれを自分の手でやる

文章を部品に割る ― トークン化

- 文章をモデルに渡す単位に割ることを**トークン化**（tokenization）、1つ1つの部品を**トークン**（token）と呼ぶ
- 割り方は選べる：

割り方	「吾輩は猫である」の例	部品の種類
単語ごと	吾輩 / は / 猫 / で / ある	数十万〜
文字ごと	吾 / 輩 / は / 猫 / で / あ / る	数十〜数千

- この講義では**文字ごと**で通す ― 部品が少なく、仕組みがぜんぶ見える

> **ノート**
>
> ChatGPT などの実物は中間の**サブワード**（subword）単位を使う ― この回の終わりに一目だけ見る。

材料：日本の市区町村名

- これから数回の材料は、**全国 1,747 市区町村の名前の「よみがな」**
- **chimei.csv** をダウンロードして、作業フォルダの **data/** に置く

Code

```
import pandas as pd

names = pd.read_csv("data/chimei.csv")["yomi"].tolist()
print(len(names), "件")
print(names[:5])
```

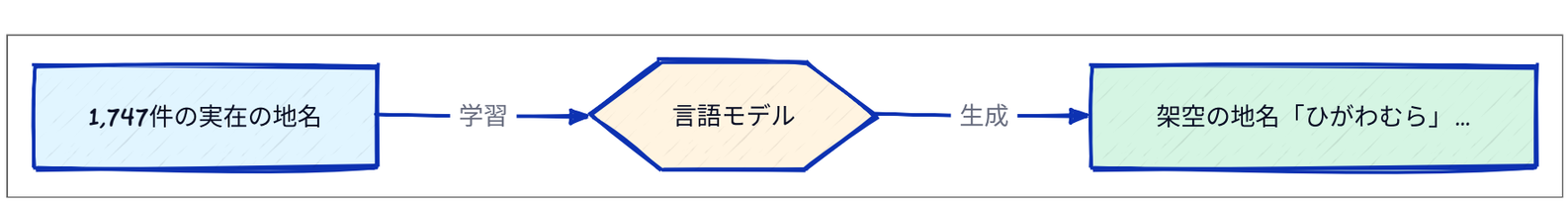
Result

```
1747 件
['あいおいし', 'あいかわまち', 'あいさいし', 'あいしょうちょう', 'あいずみちょう']
```

> ⓘ **出典**
>
> 総務省「全国地方公共団体コード」より作成（政府標準利用規約）。カタカナ表記をひらがなに変換してある。

この材料で何をするか

- ゴール：この名前たちを読み込んで、**実在しない「架空の市町村名」を作る機械**を育てる
- 名前は短い・ひらがなだけ・でも「らしさ」がある — 練習台として最高の性質



- どこまで「本物っぽく」なるかは、これから作るモデルの腕次第

語彙 — 使う文字の一覧表

- データに出てくる文字を全部集めた集合を **語彙**（**vocabulary**）と呼ぶ

Code

```
import pandas as pd

names = pd.read_csv("data/chimei.csv")["yomi"].tolist()
chars = sorted(set("".join(names)))   # 全部つなげて、重複を消して、並べる
print("語彙サイズ:", len(chars))
print("".join(chars))
```

Result

```
語彙サイズ： 74
あいうえおかがきぎくぐけげこごさざしじすずせぜそぞただちぢっつづてでとどなにぬねのはばぱひびぴふぶぷへべ
ぺほぼぽまみむめもゃやゅゆょよらりるれろわん
```

- ひらがな74種類だけ — 「ゑ」も「ー」も地名には出てこない
- **語彙の1文字1文字に、この並び順で番号を振る**

番号との対応表を作る

- 文字→番号の辞書と、番号→文字のリスト。この2つで変換は両方向できる

Code

```
import pandas as pd

names = pd.read_csv("data/chimei.csv")["yomi"].tolist()
chars = sorted(set("".join(names)))

stoi = {c: i for i, c in enumerate(chars)}   # string to int
itos = chars                                  # int to string（リストのまま使える）
print(stoi["あ"], stoi["ん"])
print(itos[0], itos[73])
```

Result

```
0 73
あ ん
```

- **stoi** / **itos** という変数名はこの先の回でもずっと使う

encode と decode

Code

```python
import pandas as pd

names = pd.read_csv("data/chimei.csv")["yomi"].tolist()
chars = sorted(set("".join(names)))
stoi = {c: i for i, c in enumerate(chars)}
itos = chars

def encode(s):
    return [stoi[c] for c in s]

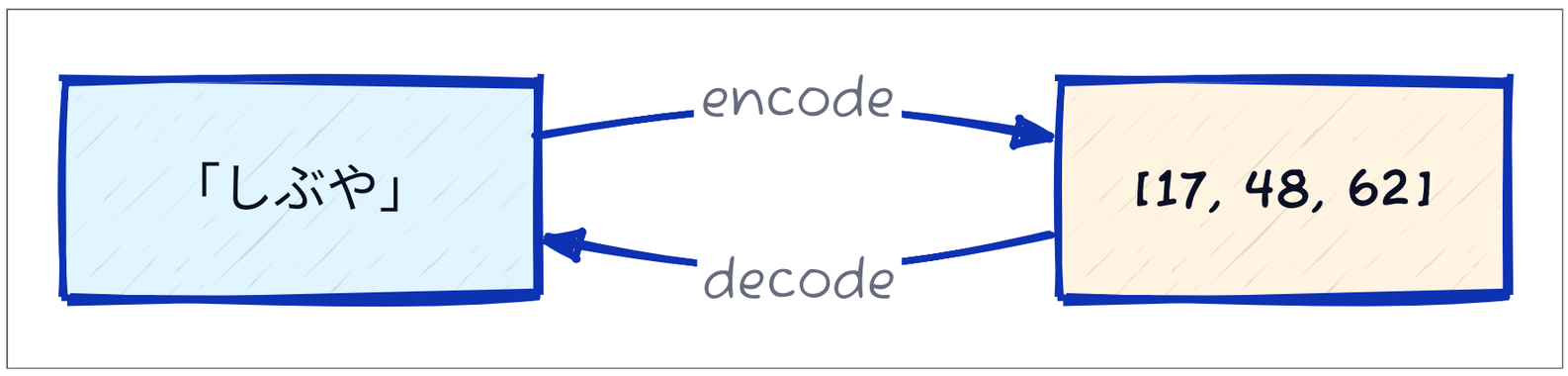
def decode(ids):
    return "".join(itos[i] for i in ids)

print(encode("しぶや"))
print(decode(encode("しぶや")))
```

Result

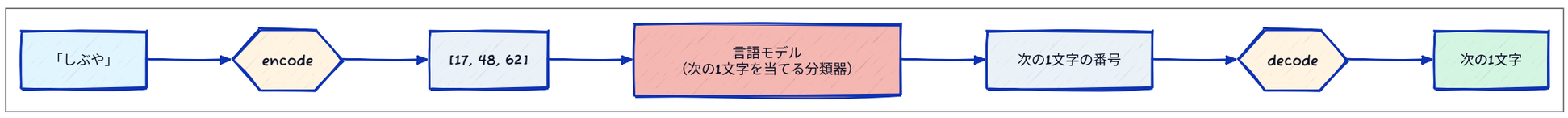
```
[17, 48, 62]
しぶや
```

- 行って戻ってこられる（**往復できる**）ことが対応表の合格条件



対応表は、機械の入り口と出口

- 完成形の中で **encode** / **decode** がどこに立つか、先に見ておく：



- モデルの中を流れるのは**番号だけ**。「し」の字の形も意味も、モデルは一生見ない
- **encode** は玄関、**decode** は出口 — この2つの**外側**だけが人間の世界
- ただし番号はまだ「ただの整理番号」— 41番と42番が似ているわけではない。番号に**意味**を持たせる工夫は「ことばの地図」の回で

演習：往復させる

自作の **encode** / **decode** が往復できることを確かめよ。

0. さっきのスライドのコードを **.py** ファイルにコピーして実行
1. 好きな市区町村名（ひらがな）を **encode** して番号列を表示
2. その番号列を **decode** して元に戻ることを確認（**==** で比較した結果も表示）
3. 「とうきょう」のように**データに実在しない並び**でも往復できることを確認

▸ 解答例を見る

表に無い文字が来たら？

ⓘ クイズ

encode("パリ") を実行したら何が起きる？予想してから実行。

- **KeyError: 'パ'** — **語彙に無い文字は変換できずにエラー**で止まる
- これは正しい挙動。だまって握りつぶすより、その場で止まって教えてくれる方がいい
- 実物の LLM も同じ問題を抱えている：語彙にどんな部品を揃えるかは設計の急所

語彙の外と、どう付き合うか

ⓘ クイズ

ChatGPT に絵文字や顔文字を打ち込んでも、KeyError では止まらない。どう解決していると思う？

- 語彙に無い文字への作戦は、だいたい3択：

作戦	中身	代償
黙って捨てる	知らない文字を無視して続行	**入力が化ける（最悪）**
「その他」の枠	未知の文字を全部、**未知語トークン**（unknown token）1つに置き換え	情報が消える
部品を細かくする	バイト単位まで割れば**何でも**表せる	列が長くなる

- 実物の答えは3つ目：GPT-2 以降の語彙は**バイトまで割れる部品を常備** — 地球上のどんな文字列も語彙外にならない
- 自作の **encode** が KeyError で**止まる**のも立派な設計 — 黙って捨てるくらいなら、その場で叫ぶ

大きな文章だと語彙はどうなる？

- 最終回に使う材料も、ここで一目だけ：夏目漱石**『吾輩は猫である』全文**
- **neko.txt** をダウンロードして **`data/`** に置く

クイズ

『吾輩は猫である』全文（約32万字）に出てくる文字は**何種類**？ ひらがな＋カタカナで約170、そこに漢字が乗る。1000？ 2000？

Code

```
text = open("data/neko.txt", encoding="utf-8").read()
print("総文字数:", len(text))
print("語彙サイズ:", len(set(text)))
```

Result

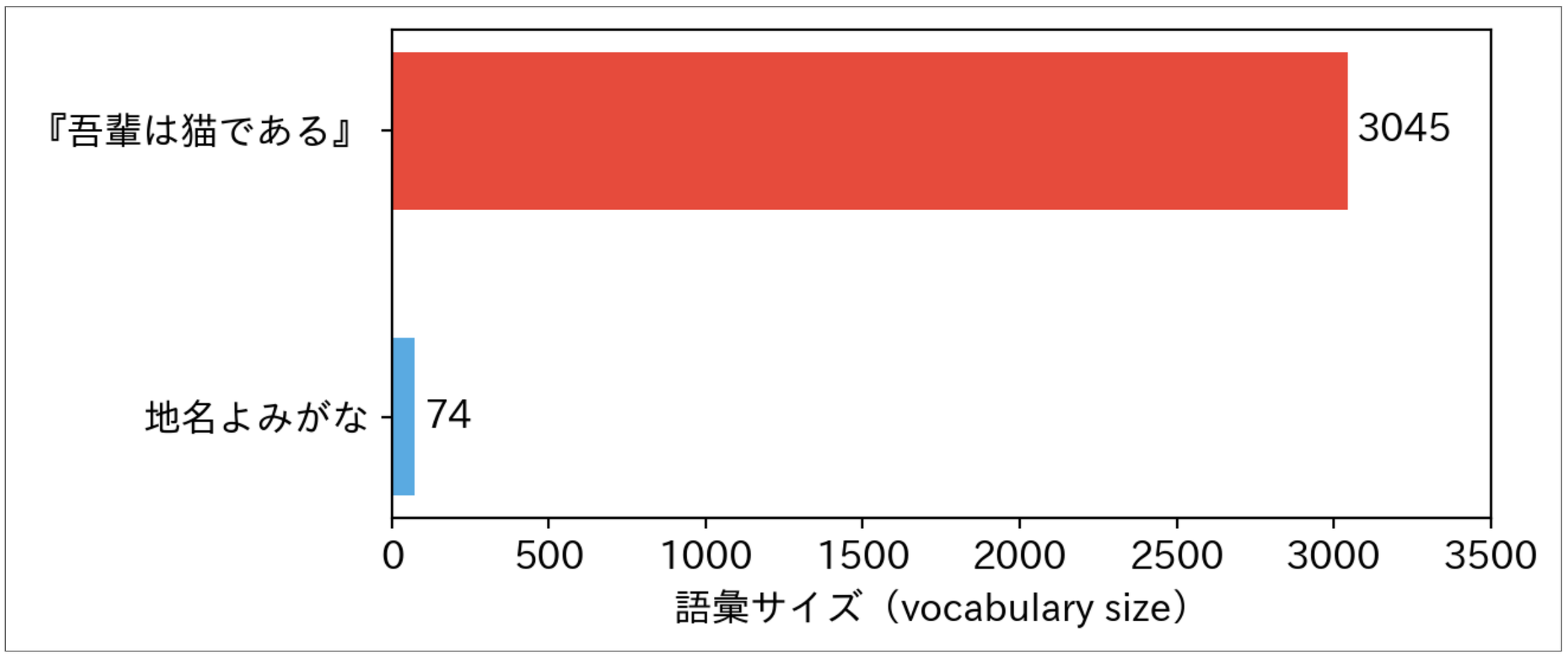
```
総文字数: 321139
語彙サイズ: 3045
```

- **3,000超** — 日本語の文字レベル言語モデルは、この大語彙と戦うことになる

出典

青空文庫『吾輩は猫である』（著作権消滅済み）。ルビ・注記を除去したテキスト。

語彙サイズ比較



- 同じ「日本語」でも、材料しだいで語彙は**40倍**違う
- 語彙が大きい＝分類の選択肢が多い＝当てるのが難しい。まずは74択の地名から

第3の割り方 — サブワード

- 単語ごとは種類が多すぎ、文字ごとは『猫』でも3,045種。実物は**中間**を取る：

割り方	「吾輩は猫である。」	種類
単語ごと	吾輩 / は / 猫 / で / ある / 。	数十万〜
サブワード	吾輩は / 猫 / である。	**数万**
文字ごと	吾 / 輩 / は / 猫 / で / あ / る / 。	数十〜数千

- 作り方は意外なほど素朴：**よく隣り合う2文字を数えて、合体**を繰り返すだけ（**BPE** と呼ぶ）
- 『猫』の前半10万字で実際に回すと、合体20回で「吾輩」「主人」が部品になり、100回で「である。」が丸ごと1部品になる
- 本編は最後まで文字ごとで通す。合体の職人技は回数外の **BPE の付録** で1から自作できる

まとめ

- ことばを数にする手順：**トークン化**（文字ごとに割る）→ **語彙**（文字一覧）→ 番号の対応表
- **`stoi`** / **`itos`** / **`encode`** / **`decode`** — この4点セットはこの講義の全域で使い回す
- 語彙に無い文字はエラー — 実物はバイトまで割って逃がす。地名は74種類、『猫』は3,000超、実物のサブワードは数万

現在の設計図　文章 → **文字に分割 ⇒ 番号** → モデル → 番号 → 文字

ⓘ 次回予告

数になったので、いよいよ「次の1文字を当てる」が始められる。 最初のモデルは学習すらしない — **数えるだけ**。それでも名前が作れてしまう。

練習問題

以下の問題を **Pythonコード** で解け。

問題1：一番よく使われる文字

地名データ全体（1,747件をつなげた文字列）で、**登場回数の多い文字トップ5** を回数つきで表示せよ。

1. **chimei.csv** を読み込み、`"".join(...)` で1本の文字列に
2. 各文字の回数を辞書で数える（`collections.Counter` を使ってもよい）
3. 多い順に並べてトップ5を表示

問題2：使われていないひらがな

五十音の基本46文字（**"あいうえおかきくけこさしすせそたちつてとなにぬねのはひふへほまみむめもやゆよらりるれろわをん"**）のうち、**地名のよみに1度も使われていない文字** をすべて表示せよ。

問題3：秘密のメッセージ

本編の対応表（`stoi` / `itos`）で、次の番号列を `decode` せよ。

```
[66, 59, 25, 67, 14, 2, 5, 9, 30, 8, 41, 56, 27, 40, 36, 56, 3, 26]
```

読めたら、その文はこの講義の **次回予告** になっている。

解答例

▸ 解答例を見る

Result

```
し 1116
ち 860
う 730
ま 593
ょ 514
未使用: ['を']
よめたらごうかくつぎはまちのなまえだ
```